

ках — языках программирования. С грамматикой некоторых языков (Рапиры, Паскаля) вы уже, наверное, знакомы, ну а какие обороты в каких случаях надо применять? Вот на этот вопрос, а заодно и на вопрос, как понимает ЭВМ неправильные обороты, мы и попытаемся дать ответ.

Подготовительные задачи к уроку 1

1. Дано число x . Вычислить приближенное значение $\sqrt{x}$ с точностью ε с помощью метода последовательных приближений:

$$y_0 = \frac{x}{2}, \quad y_i = \frac{1}{2} \left(y_{i-1} + \frac{x}{y_{i-1}} \right), \quad i \geq 1$$

Вычисления вести до тех пор, пока не станет $|y_i - y_{i-1}| < \varepsilon$.

2. Напечатать значения первых N чисел Фибоначчи¹

$$f_0 = 1, f_1 = 1, f_i = f_{i-1} + f_{i-2}, i \geq 2.$$

3. Даны массив коэффициентов $a_i, i = 0, \dots, n$, и значение x . Вычислить значение многочлена

$$y = \sum_{i=0}^n a_i x^i \text{ применяя формулу:}$$

$$y = (\dots (a_n x + a_{n-1})x + \dots + a_1)x + a_0$$

(Этот очень эффективный прием вычисления многочленов называется *схемой Горнера*: в нем n сложений, n умножений и ни одного (!) возведения в степень.)

4. Дано x . Вычислить приближенное значение $\sin x$ с точностью ε , пользуясь формулой

$$\sin x \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots + (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

Считать известным, что точность ε будет достигнута, когда

$$\left| \frac{x^{2n+1}}{(2n+1)!} \right| < \varepsilon.$$

В задачах 1, 2 не использовать массивы. В задаче 4 не использовать возведение в степень.

После решения задач внимательно сравните условия и решения всех задач, и попробуйте ответить на вопросы:

- 1) что общего в условиях этих задач?
- 2) что общего в их решениях?

Сравните ваши решения с приведенными на с. 54. Задача 3, наверное, трудностей не вызвала. Если в задачах 1, 2, 4 ваши решения построены на тех же идеях, что и решения а), то вы сделали все правильно, а если на идеях решений б), то вы ошиблись. Конечно, и программы б) будут работать и дадут в большинстве случаев нужный вам результат, и все же любой опытный программист скажет, что они не годятся. Почему?

Для ответа на этот вопрос давайте посмотрим, что общего в условиях и решениях задач. Вы заметили, наверное, и часто пользоваться определенными операциями и переписывания значений (обведены рамочкой). Но самым главным является то, что **оче-**

редные значения вычисляются через предыдущие (см. подчеркнутые части программ). В математике зависимость новых значений от старых называется *рекуррентной*, а число старых значений, от которых зависит очередное значение, называется *порядком* рекуррентной формулы.

Таким образом, в задачах 1 и 2 мы имели рекуррентные формулы соответственно 1-го и 2-го порядка, а в задачах 3 и 4 мы имели неперекрестную запись многочлена и формулы (1) к рекуррентной:

$$y = \sum_{i=0}^n a_i x^i \Rightarrow y_0 = a_n, \quad y_i = y_{i-1}x + a_{n-i}$$

$$y = \sum_{i=0}^n (-1)^i \frac{x^{2i+1}}{(2i+1)!} \Rightarrow y_0 = 0,$$

$$z_0 = x; \quad y_i = y_{i-1} + z_{i-1}; \quad z_i = \frac{-z_{i-1}x^2}{2i(2i+1)}.$$

Теперь взглянем на решения б) и зададим себе вопрос: если бы вы вручную считали по формуле (1), то разве стали бы вы вычислять каждый раз степень или считать факториал заново? Вряд ли. А потому нужно использовать рекуррентные варианты, а это значит, что нужно хранить промежуточные значения. А в задаче 1 и вообще неясно, сколько этих промежуточных значений будет и какой объем памяти под них надо отвести. Кроме того, оказывается, что для ЭВМ операции индексирования — это очень трудоемкая операция: более трудоемкая, чем, например, умножение (вообще «точка зрения» ЭВМ на трудоемкость отдельных операций совсем не совпадает с человеческой). Теперь вам стали ясны запреты в условиях задач: мы запретили лишний расходовать память и использовать трудоемкие операции. Современные ЭВМ имеют огромную память и колоссальную скорость, и все же есть очень много задач, для которых скорости и памяти не хватает. Поэтому программист должен уметь бережно расходовать ресурсы ЭВМ и, следовательно, решения б) нельзя признать пригодными.

Откуда же появляются такие ошибки? Дело в том, что программист часто переводит алгоритм с языка математики на язык программирования. Эти языки похожи, но правила «разговора» на них различны. Поэтому «подстрочный» перевод с языка математики на язык программирования может дать совсем неправильный результат, также как дословный перевод русского приветствия «Добрый день!» на английский — «Kind day!» — ошибочен: для англичанина это не приветствие, а бессмыслица.

Возвращаясь к нашим задачам, отметим разницу в «правилах разговора» на языке математики и языке программирования. *Математики используют индексацию для различных значений переменных как «в пространстве», так и «во времени», программисты — только для различения в пространстве.* В рекуррентных формулах индексы характеризуют время (номер шага) появления очередного значения, поэтому при программировании они не нужны: новые значения просто засылаются на старое место.

¹Об этих замечательных числах можно прочитать в «Кванте» 1979, №5, с. 53 или в брошюре Н. Н. Воробьева «Числа Фибоначчи» (М., «Наука», 1978), но для решения задачи это требуется

Решения подготовительных задач к уроку 1

(Описания и ввод-вывод опущены, приведены только тела программ)	
(a)	(б)
Задача 1 (Квадратный корень)	
<pre> y := x/2; repeat y0 := y; y := 0.5 * (y0 + x/y0) until abs(y0 - y1) < eps; </pre>	<pre> y[0] := x/2; i := 0; repeat i := i + 1; y[i] := 0.5 * (y[i-1] + x/y[i-1]) until abs(y[i] - y[i-1]) < eps; </pre>
Задача 2 (Числа Фибоначчи)	
<pre> f2 := 1; f1 := 1; for i := 3 to n do begin f := f2 + f1; writeln(f); f2 := f1; f1 := f end; </pre>	<pre> f[0] := 1; f[1] := 1; for i := 3 to n do begin f[i] := f[i-1] + f[i-2]; writeln(f[i]) end; </pre>
Задача 3 (Схема Горнера)	
<pre> y := a[n]; for i := n-1 downto 0 do y := y * x + a[i]; </pre>	
Задача 4 (Вычисление синуса)	
<pre> s := 0; элем := x; k := 1; while abs(элем) > eps do begin s := s + элем; k := k + 2; элем := -элем * x * x / (k * (k-1)) end; </pre>	<pre> s := 0; элем := x; k := 0; while abs(элем) > eps do begin s := s + элем; k := k + 1; факт := 1; for i := 2 to 2*k+1 do факт := факт * i; факт := степ(-1,k) * степ(x,2*k+1) / факт {функция степ(a,n) даёт значение a^n } end; </pre>

Таким образом:

При счете по рекуррентной формуле k -го порядка достаточно иметь $k+1$ дополнительный элемент памяти для хранения последних k значений и очередного значения.

В рекуррентных формулах 1-го порядка, если не надо сравнивать соседние значения, можно обойтись вообще одним элементом (см. задачу 3).

Программы, реализующие рекуррентные соотношения, включают в себя следующие фрагменты:

- начальные присваивания (в решениях — это первые строчки)
- основной цикл счета по формулам,
- в теле цикла: вычисление очередного значения, использование этого значения и переприсваивание

Эти элементы имеются обязательно, хотя выглядеть могут по-разному. Начальные присваивания могут выполняться по сложному алгоритму. Основной цикл может быть с заранее известным (как в задачах 2, 3) или заранее неизвестным числом повторений (как в задачах 1, 4). В формулах 1-го порядка переписывание может быть совмещено с вычислением очередного значения. Очередное значение может использоваться в дальнейших расчетах (как значение *элем* в задаче 4), просто печататься (задача 2) или

не использовать вообще для чего-либо иного, кроме получения следующего значения.

В заключение рассмотрим задачу с рекуррентной формулой k -го порядка: *напечатать первые N значений, определяемых рекуррентными формулами:*

$$f_i = 1, f \leq k; f_{i+1} = (f_{i-k+1} + f_{i-k+2} + \dots + f_{i-1})f_i. \quad i \geq k$$

Для решения нам понадобится массив в $k+1$ элемент для размещения последних k значений и нового значения. Обратите внимание: этот массив работает как трамвай: очередное значение «входит» в него через переднюю дверь (а не в заднюю, как мы привыкли), «проезжает» k остановок, проваливаясь назад,

